Python Complete Notes

Beginner to Advanced — Simple Language Madhe

Python Syllabus Guide

2026

Contents

1 Introduction — Ha Notes Kasa Vaparaycha **4**

2 PART 1: PYTHON INTRODUCTION **5**
- 2.1 1.1 Python Kay Aahe? . . . 5
- 2.2 1.2 Installation . . . 5
- 2.3 1.3 VS Code Setup . . . 6
- 2.4 1.4 Variables . . . 6
- 2.5 1.5 Data Types . . . 7
- 2.6 1.6 Input / Output . . . 7
- 2.7 1.7 Operators . . . 8
- 2.8 1.8 Type Casting . . . 9
- 2.9 1.9 Strings (Complete) . . . 10

3 PART 2: DATA STRUCTURES AND CONTROL FLOW **12**
- 3.1 2.1 Lists . . . 12
- 3.2 2.2 Tuples . . . 13
- 3.3 2.3 Sets . . . 13
- 3.4 2.4 Dictionary . . . 14
- 3.5 2.5 if-else (Conditional Statements) . . . 14
- 3.6 2.6 Loops . . . 15
- 3.7 2.7 Pattern Programs . . . 16

4 PART 3: FUNCTIONS, FILE HANDLING AND MODULES **18**
- 4.1 3.1 Functions . . . 18
- 4.2 3.2 Lambda Functions . . . 19
- 4.3 3.3 Recursion . . . 19
- 4.4 3.4 File Handling . . . 20
- 4.5 3.5 Exception Handling . . . 21
- 4.6 3.6 Modules . . . 22

5 PART 4: OOP (OBJECT ORIENTED PROGRAMMING) COMPLETE **24**
- 5.1 4.1 OOP Introduction — Class ani Object . . . 24
- 5.2 4.2 Inheritance . . . 24
- 5.3 4.3 Polymorphism . . . 26

5.4 4.4 Encapsulation . . . 27
5.5 4.5 Abstraction . . . 27
5.6 4.6 Decorators . . . 28
5.7 4.7 Generators . . . 29

6 Summary — Pudhe Kay KarAyache? 31

7 PART 5: PRACTICE EXERCISES WITH SOLUTIONS 32
7.1 5.1 Variables and Data Types — Practice . . . 32
7.2 5.2 Operators and Type Casting — Practice . . . 32
7.3 5.3 Strings — Practice . . . 33
7.4 5.4 Lists, Tuples, Sets, Dictionary — Practice . . . 33
7.5 5.5 Loops and Patterns — Practice . . . 34
7.6 5.6 Functions, Lambda, Recursion — Practice . . . 35
7.7 5.7 File and Exception Handling — Practice . . . 35
7.8 5.8 OOP — Practice . . . 36

8 PART 6: MINI PROJECTS (FULL CODE) 38
8.1 6.1 Project 1 — Simple Calculator . . . 38
8.2 6.2 Project 2 — To-Do List (using Lists) . . . 38
8.3 6.3 Project 3 — Number Guessing Game . . . 39
8.4 6.4 Project 4 — Student Management System (Full OOP) . . . 40

9 PART 7: COMMON INTERVIEW QUESTIONS (Q&A) 42
9.1 7.1 Basics . . . 42
9.2 7.2 Data Structures . . . 42
9.3 7.3 Functions and OOP . . . 42
9.4 7.4 Advanced Concepts . . . 43

10 PART 8: QUICK REFERENCE CHEAT SHEET 44
10.1 8.1 Common Built-in Functions . . . 44
10.2 8.2 String Methods Quick List . . . 44
10.3 8.3 List Methods Quick List . . . 44
10.4 8.4 OOP Quick Reference . . . 45
10.5 8.5 Exception Handling Quick Syntax . . . 45

11 Final Note 46

12 PART 9: BONUS TOPICS (EXTRA DEPTH) 47
12.1 9.1 Global vs Local Variables . . . 47
12.2 9.2 Comments in Python . . . 47
12.3 9.3 The `zip()` and `enumerate()` Functions . . . 48
12.4 9.4 The Walrus Operator `:=` . . . 48
12.5 9.5 Ternary (Conditional) Expression . . . 48
12.6 9.6 Common Errors and How to Debug Them . . . 48
12.7 9.7 String Formatting Deep Dive . . . 49
12.8 9.8 Additional Practice Problems (Mixed) . . . 49

13 PART 10: MORE PRACTICE PROBLEMS (ADVANCED SET) 52

14 PART 11: MORE MINI PROJECTS **56**
14.111.1 Project 5 — Contact Book (Dictionary + File Storage) 56
14.211.2 Project 6 — Quiz Game (OOP + Score Tracking) 57
14.311.3 Project 7 — Library Management System (Full OOP with Inheritance) 57
14.411.4 Project 8 — Inventory Management System 59

15 PART 12: USEFUL BUILT-IN MODULES DEEP DIVE **61**
15.112.1 `datetime` Module — Dates and Times 61
15.212.2 `json` Module — Working with JSON Data 61
15.312.3 `re` Module — Regular Expressions (Basics) 62
15.412.4 `os` Module — File and Directory Operations 62
15.512.5 `random` Module — Random Values 62
15.612.6 `collections` Module — Useful Data Structures 63
15.712.7 `itertools` Module — Advanced Looping Tools 63

16 PART 13: MEGA PROJECT — HOSPITAL MANAGEMENT SYSTEM (Complete OOP) **64**

17 PART 14: QUICK-FIRE QUESTIONS (MCQ & TRUE/FALSE) **67**

1 Introduction — Ha Notes Kasa Vaparaycha

Namaskar! Hi PDF tumchya sathi banavli aahe — pura Python syllabus, ekdum simple ani sopya bhashet, step by step. Prattek topic madhe teen goshti astIla:

1. **Concept kay aahe** — sopya shabdat samjaun sangitale.
2. **Syntax / Code example** — actual Python code, run karun baghU shakatA.
3. **Output ani important points** — lakshat thevNyAsArakhyA goShTI, common mistakes.

Ha document 4 bhagant vibhaglA aahe:

- **Part 1**: Python Introduction (Installation te Strings)
- **Part 2**: Data Structures (Lists, Tuples, Sets, Dictionary) ani Control Flow (if-else, Loops, Patterns)
- **Part 3**: Functions, Lambda, Recursion, File Handling, Exception Handling, Modules
- **Part 4**: OOP Complete (Classes, Inheritance, Polymorphism, Encapsulation, Abstraction, Decorators, Generators)

Chala, suru karUyA!

2 PART 1: PYTHON INTRODUCTION

2.1 1.1 Python Kay Aahe?

Python ek **high-level, general-purpose programming language** aahe. YachA arth asA ki tumhi Python madhe English sarkhya sopya words vAparUna code lihu shakta, ani computer to samjun ghenyasathi machine language madhe convert karto.

Python popular ka aahe:

- **Sopi syntax** — English sarkhi vachnyat yete.
- **Free ani Open Source** — koNIhI free madhe download karu shakto.
- **Versatile** — Web development, Data Science, AI/ML, Automation, Games, sagaḷikade vaparla jato.
- **Huge community** — problem ala tar Google/StackOverflow var lakho solutions miltat.
- **Cross-platform** — Windows, Mac, Linux sagaḷikade chalte.

Python ha language 1991 madhe **Guido van Rossum** ne banavla hota.

2.2 1.2 Installation

Python code run karnyasathi tumchya computer var Python installed asne garjeche aahe.

2.2.1 Windows var Installation

1. Browser madhe `python.org` website open kara.
2. **Downloads** section madhe ja, latest version (jase Python 3.12+) download kara.
3. Installer file open kara. **VERY IMPORTANT**: installer che pahilya screen var khali "**Add Python to PATH**" cha checkbox asato — to nakki tick kara. Nahi tar terminal madhe `python` command work karNaar nahi.
4. "Install Now" var click kara ani install hoNyAchI wait kara.
5. Installation nantar terminal (cmd) open karun khAlIla command taka:

```
python --version
```

YAne Python cha version disalA tar installation successful zale aahe.

2.2.2 Mac/Linux var Installation

Mac ani Linux madhe bahutek veLaa Python already installed asto. Check karnyasathi terminal madhe:

```
python3 --version
```

Nasel tar Mac var **Homebrew** vaparun install kartA yete: `brew install python3`. Linux var (Ubuntu) `sudo apt install python3` command vaparlI jate.

2.3 1.3 VS Code Setup

VS Code (Visual Studio Code) ha ek free code editor aahe je Python code lihNyaasathi sarvAta jast vaparle jate.

2.3.1 Steps:

1. `code.visualstudio.com` varun VS Code download ani install kara.
2. VS Code open kara, DAvyA baju la **Extensions** icon (chaukonI box sarkha) var click kara.
3. Search box madhe "**Python**" taka — Microsoft cha official Python extension install kara.
4. Ekhate `.py` file banvA (jase `hello.py`).
5. Code lihA ani top-right cornar madhe **Run (▶)** button dAbA, kiMvA terminal madhe run kara:

```
python hello.py
```

Tip: VS Code madhe terminal open karnyasathi 'Ctrl + " (backtick) shortcut vaparava.

2.4 1.4 Variables

Variable ha ek **container/box** aahe jyat aapan data store karto. Python madhe variable banvaNyaasathi kuthlAhI special keyword lagat nahi — sraLa nAva deUna value assign kara.

```
name = "Rahul"
age = 21
height = 5.9

print(name)
print(age)
print(height)
```

Output:

```
Rahul
21
5.9
```

2.4.1 Variable Naming Rules:

- Nava letter (a-z, A-Z) kiMvA underscore (_) ne suru vhAve pahije.
- Numbers ne suru hou shakta nahi (`1name` chalnar nahi, `name1` chalel).
- Spaces vaparta yet nahit — `first name` aivajI `first_name` vapara.
- Python madhe special keywords (`class`, `if`, `for`, i.) variable name mhaNUna vaparu shakta nahi.
- Case-sensitive aahe — `Name` ani `name` ha vegaLe variable manle jAtAta.

2.4.2 Multiple Assignment:

```
a, b, c = 10, 20, 30
print(a, b, c)   # 10 20 30

x = y = z = 100
print(x, y, z)   # 100 100 100
```

2.5 1.5 Data Types

Python madhe data che multiple types astat — pratyek type eka vegLyA prakaracha data store karto.

Data Type	Example	Meaning
`int`	`10, -5, 200`	Whole numbers
`float`	`10.5, -3.14`	Decimal numbers
`str`	`"Hello"`	Text/String
`bool`	`True, False`	Boolean value
`list`	`[1, 2, 3]`	Ordered, changeable collection
`tuple`	`(1, 2, 3)`	Ordered, unchangeable collection
`set`	`{1, 2, 3}`	Unordered, unique items
`dict`	`{"a": 1}`	Key-value pairs
`NoneType`	`None`	Empty/no value

Data type check karnyasathi `type()` function vaparte:

```
x = 10
y = "Hello"
z = 3.14
w = True

print(type(x))   # <class 'int'>
print(type(y))   # <class 'str'>
print(type(z))   # <class 'float'>
print(type(w))   # <class 'bool'>
```

Important: Python ek **dynamically typed** language aahe — mhaNaje variable la data type declare karAyachI garaj nahi, Python ApoApa value baghUna type Tharavate.

2.6 1.6 Input / Output

2.6.1 Output — `print()`

`print()` function screen var kahi tari display karnyasathi vaparle jate.

```
print("Hello, World!")
print("Sum is:", 10 + 20)
print("A", "B", "C", sep=" - ")     # A - B - C
print("No newline", end=" ")
print("continues here")
```

2.6.2 Input — `input()`

`input()` function user kaDUna keyboard varun data ghenyasathi vaparle jate. **Ha function nehmi string (`str`) return karto**, mhanun number pahije asalyAsa convert (typecast) karAve lagte.

```
name = input("Tumche naav kay? ")
age = int(input("Tumche vay kitI? "))

print("Namaskar", name)
print("Pudhchya varshi tumche vay", age + 1, "asel")
```

Common mistake: `input()` cha result direct number sarkha vapara try kelyasa error yeto:

```
age = input("Age: ")
print(age + 5)   # ERROR! str + int chalta nahi
```

Yasathi `int(input(...))` kiMvA `float(input(...))` vAparaNe garjeche aahe.

2.7 1.7 Operators

Operators vaparun aapan values var operations perform karto.

2.7.1 Arithmetic Operators

```
a = 10
b = 3

print(a + b)   # 13  -> Addition
print(a - b)   # 7   -> Subtraction
print(a * b)   # 30  -> Multiplication
print(a / b)   # 3.33 -> Division (float result)
print(a // b)  # 3   -> Floor Division (decimal cut)
print(a % b)   # 1   -> Modulus (remainder)
print(a ** b)  # 1000 -> Power (a to the power b)
```

2.7.2 Comparison Operators

```
print(10 == 10)   # True
print(10 != 5)    # True
```

```
print(10 > 5)     # True
print(10 < 5)     # False
print(10 >= 10)   # True
print(10 <= 9)    # False
```

2.7.3 Logical Operators

```
a = True
b = False

print(a and b)   # False -> Both True pahije
print(a or b)    # True  -> Ekhi True asel tar chalte
print(not a)     # False -> Ulta karto
```

2.7.4 Assignment Operators

```
x = 10
x += 5   # x = x + 5  -> 15
x -= 2   # x = x - 2  -> 13
x *= 3   # x = x * 3  -> 39
x /= 3   # x = x / 3  -> 13.0
```

2.8 1.8 Type Casting

Type Casting mhanje ek data type dusRyA data type madhe convert karne.

```
# str -> int
a = "10"
b = int(a)
print(b + 5)      # 15

# int -> float
x = 10
y = float(x)
print(y)          # 10.0

# float -> int (decimal katla jato, round nahi hot)
p = 9.8
q = int(p)
print(q)          # 9

# int -> str
n = 100
s = str(n)
print("Number is " + s)   # Number is 100
```

Important: `int("10.5")` direct chalta nahi — error yeto, karan string madhe decimal point aahe. Asa case madhe pahile `float()` madhe convert karava, mag `int()`.

2.9 1.9 Strings (Complete)

String mhanje characters cha collection, jo single (') kiMvA double (") quotes madhe lihlA jato.

```
name = "Python Programming"
```

2.9.1 String Indexing ani Slicing

Prattek character la ek position (index) aste, jo **0 pAsUna** suru hoto.

```
s = "PYTHON"
print(s[0])     # P
print(s[-1])    # N  (last character)
print(s[1:4])   # YTH  (index 1 te 3)
print(s[:3])    # PYT
print(s[3:])    # HON
print(s[::-1])  # NOHTYP  (reverse string)
```

2.9.2 Important String Methods

```
s = "  Hello Python World  "

print(s.upper())          # HELLO PYTHON WORLD
print(s.lower())          # hello python world
print(s.strip())          # "Hello Python World" (spaces kadhun)
print(s.replace("Python", "Java"))  # Hello Java World
print(s.split())          # ['Hello', 'Python', 'World']
print(len(s))             # total characters count
print(s.find("Python"))   # position kuthe suru hote
print("Python" in s)      # True -> substring check
print(s.count("o"))       # 'o' kitide veLaa aala
```

2.9.3 String Concatenation ani Formatting

```
first = "Rahul"
last = "Sharma"

# Concatenation
full = first + " " + last
print(full)

# f-string (best/modern way)
age = 21
```

```
print(f"My name is {first} {last} and I am {age} years old")

# format() method
print("My name is {} and I am {}".format(first, age))
```

2.9.4 String is Immutable

Python madhe string **immutable** aahe — mhanje ekada banlA ki tya string madhe direct change kartA yet nahi.

```
s = "Hello"
# s[0] = "J"   # ERROR! String madhe item assign karta yet nahi
s = "Jello"    # Nava string banvAvA lagto
```

3 PART 2: DATA STRUCTURES AND CONTROL FLOW

3.1 2.1 Lists

List ek **ordered ani changeable (mutable)** collection aahe, jyat multiple values ek variable madhe store karta yetat. Square brackets `[]` vaparle jAtAta.

```
fruits = ["apple", "banana", "mango", "grapes"]
print(fruits)
print(fruits[0])       # apple
print(fruits[-1])      # grapes
```

3.1.1 List Operations

```
fruits = ["apple", "banana", "mango"]

fruits.append("orange")        # shevaTI add karto
print(fruits)                  # ['apple','banana','mango','orange']

fruits.insert(1, "kiwi")        # specific position var add
print(fruits)

fruits.remove("banana")         # value kadhto
fruits.pop()                    # last item kadhto
fruits.pop(0)                   # index 0 cha item kadhto

fruits.sort()                   # alphabetically sort karto
fruits.reverse()                # ulta karto

print(len(fruits))              # total items count
```

3.1.2 List Slicing ani Looping

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])     # [20, 30, 40]
print(nums[:3])      # [10, 20, 30]

for n in nums:
    print(n)
```

3.1.3 List Comprehension (Advanced)

Ek line madhe list banvaNyAchI shortcut method.

```
squares = [x**2 for x in range(1, 6)]
print(squares)     # [1, 4, 9, 16, 25]
```

```python
even = [x for x in range(1, 11) if x % 2 == 0]
print(even)        # [2, 4, 6, 8, 10]
```

3.2 2.2 Tuples

Tuple pan list sarkhach aahe, pan **immutable** aahe — mhanje ekada banla ki change kartA yet nahi. Round brackets `()` vaparle jAtAta.

```python
coordinates = (10, 20)
person = ("Rahul", 21, "Pune")

print(person[0])     # Rahul
print(person[1])     # 21

# person[0] = "Amit"   # ERROR! Tuple immutable aahe
```

Tuple kadhi vaparava: jya data la change nAhI karayche (jase fixed coordinates, constant values), tya sathi tuple use karava — te list peksha fast aste.

```python
a, b, c = (1, 2, 3)   # Tuple unpacking
print(a, b, c)         # 1 2 3
```

3.3 2.3 Sets

Set ek **unordered collection** aahe jyat **duplicate values allowed naahit**. Curly brackets `{}` vaparle jAtAta.

```python
numbers = {1, 2, 3, 3, 4, 4, 5}
print(numbers)     # {1, 2, 3, 4, 5}  -> duplicates ApoApa kadhle jAtAta
```

3.3.1 Set Operations

```python
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a.union(b))         # {1,2,3,4,5,6} -> sagaḷe elements
print(a.intersection(b))  # {3,4}         -> common elements
print(a.difference(b))    # {1,2}         -> a madhe aahe pan b madhe nahi

a.add(10)          # ek item add
a.remove(1)        # item kadhto
```

Important: Set madhe items la index nasto (order nasto), mhanun `set[0]` sarkha access kartA yet nahi.

3.4 2.4 Dictionary

Dictionary madhe data **key-value pairs** swaroopat store hoto. Curly brackets `{}` vaparle jAtAta, pan `key: value` format madhe.

```
student = {
    "name": "Rahul",
    "age": 21,
    "course": "Python"
}

print(student["name"])       # Rahul
print(student.get("age"))    # 21
```

3.4.1 Dictionary Operations

```
student["age"] = 22              # value update
student["city"] = "Pune"          # nava key-value add

del student["course"]             # key kadhto

print(student.keys())             # sagaḷya keys
print(student.values())           # sagaḷya values
print(student.items())            # key-value pairs

for key, value in student.items():
    print(key, ":", value)
```

3.5 2.5 if-else (Conditional Statements)

`if-else` vaparun program ek condition check karto ani condition nusar decide karto konta code run karaycha.

```
age = 20

if age >= 18:
    print("Tumhi adult aahat")
else:
    print("Tumhi minor aahat")
```

3.5.1 if-elif-else

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 75:
```

```
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

3.5.2 Nested if

```
age = 25
has_license = True

if age >= 18:
    if has_license:
        print("Gadi chalvu shakta")
    else:
        print("License gheun ya")
else:
    print("Age kami aahe")
```

Important: Python madhe indentation (spaces) khup mahatvache aahe — {} brackets nahi vaparat, tyaaivajI indentation (4 spaces) block define karto.

3.6 2.6 Loops

Loop vaparun aapan ek code repeat karto jopariant ek condition true aahe.

3.6.1 for Loop

```
for i in range(1, 6):
    print(i)          # 1 2 3 4 5

fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
```

3.6.2 while Loop

```
i = 1
while i <= 5:
    print(i)
    i += 1        # increment karne garjeche, nahi tar infinite loop hoto
```

3.6.3 break, continue, pass

```
for i in range(1, 10):
    if i == 5:
        break          # loop pUrNa thambvato
    print(i)

for i in range(1, 10):
    if i % 2 == 0:
        continue        # ha iteration skip karto, loop continue rahto
    print(i)

for i in range(5):
    pass                 # kahi na kartA, placeholder mhanun vaparto
```

3.7 2.7 Pattern Programs

Pattern programs loops chi practice karnyasathi khup useful astat.

3.7.1 Simple Star Pattern

```
n = 5
for i in range(1, n+1):
    print("*" * i)
```

Output:

```
*
**
***
****
*****
```

3.7.2 Number Pattern

```
n = 5
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j, end=" ")
    print()
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

3.7.3 Pyramid Pattern

```
n = 5
for i in range(1, n+1):
    print(" " * (n-i) + "*" * (2*i - 1))
```

Output:

```
    *
   ***
  *****
 *******
*********
```

4 PART 3: FUNCTIONS, FILE HANDLING AND MODULES

4.1 3.1 Functions

Function ek **reusable block of code** aahe jo ek specific kaam karto. Baar baar toch code lihNyAaivajI, function banvaun to jithe lAgela tithe call karta yeto.

```
def greet(name):
    print(f"Namaskar, {name}!")

greet("Rahul")     # Namaskar, Rahul!
greet("Sneha")     # Namaskar, Sneha!
```

4.1.1 Function with Return Value

```
def add(a, b):
    result = a + b
    return result

total = add(10, 20)
print(total)     # 30
```

4.1.2 Default Parameters

```
def greet(name, msg="Good Morning"):
    print(f"{msg}, {name}")

greet("Amit")                 # Good Morning, Amit
greet("Amit", "Good Night")   # Good Night, Amit
```

4.1.3 *args and **kwargs

Jevha function la kitI arguments yenar he mahit naste, tevha `*args` (multiple values) ani `**kwargs` (multiple key-value) vaparle jAtAta.

```
def total_sum(*numbers):
    return sum(numbers)

print(total_sum(1, 2, 3, 4))   # 10

def show_info(**data):
    for key, value in data.items():
        print(key, ":", value)

show_info(name="Rahul", age=21, city="Pune")
```

4.2 3.2 Lambda Functions

Lambda ek **anonymous (naav nsalele) function** aahe, jo ek line madhe lihlA jato. Chota, simple kaam sathi vaparle jate.

```
# Normal function
def square(x):
    return x * x

# Lambda function (samach kaam)
square = lambda x: x * x

print(square(5))    # 25
```

4.2.1 Lambda with map, filter, sorted

```
nums = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x**2, nums))
print(squares)          # [1, 4, 9, 16, 25]

even = list(filter(lambda x: x % 2 == 0, nums))
print(even)             # [2, 4]

students = [("Rahul", 85), ("Amit", 92), ("Sneha", 78)]
sorted_students = sorted(students, key=lambda x: x[1], reverse=True)
print(sorted_students)  # sorted by marks (highest first)
```

4.3 3.3 Recursion

Recursion mhanje ek function jo **swtaHlAcha call karto**, ek chota problem sodvaNyaasathi.

```
def factorial(n):
    if n == 0 or n == 1:      # base condition - ithe recursion thambte
        return 1
    else:
        return n * factorial(n - 1)   # function swtaHlA call karto

print(factorial(5))     # 120  -> 5*4*3*2*1
```

Kase kaam karto (factorial(3)):

```
factorial(3) = 3 * factorial(2)
             = 3 * (2 * factorial(1))
             = 3 * (2 * 1)
             = 6
```

Important: Prattek recursive function la ek **base condition** pahije, nahi tar function

infinite veLA call hota rahil ani “RecursionError” yeIl.

4.3.1 Fibonacci Series using Recursion

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

for i in range(7):
    print(fibonacci(i), end=" ")   # 0 1 1 2 3 5 8
```

4.4 3.4 File Handling

File Handling vaparun aapan `.txt`, `.csv` sarkhya files madhe data read/write karto.

4.4.1 File Modes

Mode	Meaning
`"r"`	Read (default)
`"w"`	Write (existing content overwrite hote)
`"a"`	Append (shevaTI add hote)
`"r+"`	Read and Write

4.4.2 Writing to a File

```
file = open("data.txt", "w")
file.write("Hello Python!\n")
file.write("File handling shikat aahot.")
file.close()      # file band karne garjeche
```

4.4.3 Reading a File

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

4.4.4 Best Practice: `with` Statement

`with` vaparle tar file automatically close hote — `close()` explicitly call karaychI garaj nahi.

```python
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# File automatically closed here

with open("data.txt", "a") as file:
    file.write("\nAnkhi ek line add keli.")

with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())    # line by line read
```

4.5 3.5 Exception Handling

Jevha program run hotana ek unexpected error (exception) yeto, tevha program crash hoto. Exception Handling vaparun aapan ha error handle karto, jyAmuLe program crash na hotA smoothly chalta rahto.

```python
try:
    num = int(input("Number taka: "))
    result = 10 / num
    print(result)
except ZeroDivisionError:
    print("Zero ne divide kartA yet nahi!")
except ValueError:
    print("Kripya valid number taka!")
else:
    print("Kahi error ala nahi, sagaḷe barobar zale")
finally:
    print("Ha block nehmi run hoto, error yevaDhA kahIhI")
```

4.5.1 Common Exceptions

Exception	Kadhi yeto
`ZeroDivisionError`	Number la 0 ne divide kelyavar
`ValueError`	Chukicha value convert karnyacha prayatna kelyavar
`TypeError`	Chukicha data type operation kelyavar
`IndexError`	List/String cha index out of range asel tar
`KeyError`	Dictionary madhe naslelI key access kelyavar
`FileNotFoundError`	File sApa.Daali nahi tar

4.5.2 Custom Exception (Raise)

```python
def check_age(age):
    if age < 0:
```

```
        raise ValueError("Age negative nasavi!")
    print("Valid age:", age)

try:
    check_age(-5)
except ValueError as e:
    print("Error:", e)
```

4.6 3.6 Modules

Module mhanje ek `.py` file jyat functions, classes, variables lihle jAtAta, ani te dusRyA program madhe **import** karUna vaparle jAtAta.

4.6.1 Built-in Modules

```
import math
print(math.sqrt(25))      # 5.0
print(math.pi)            # 3.14159...

import random
print(random.randint(1, 10))    # random number 1 te 10 dramyAna

import datetime
print(datetime.datetime.now())  # current date ani time
```

4.6.2 Import Karnyachya Padhati

```
import math
print(math.sqrt(16))

from math import sqrt
print(sqrt(16))            # ata 'math.' lihNyAchI garaj nahi

from math import sqrt as sq
print(sq(16))               # short naav vaparle

import math as m
print(m.pi)
```

4.6.3 Custom Module BanvaNae

Suppose `mymodule.py` nava cha file aahe:

```
# mymodule.py
def greet(name):
    return f"Hello, {name}!"
```

```
pi_value = 3.14159
```

Doosraya file madhe (`main.py`) vapara:

```
# main.py
import mymodule

print(mymodule.greet("Rahul"))    # Hello, Rahul!
print(mymodule.pi_value)          # 3.14159
```

5 PART 4: OOP (OBJECT ORIENTED PROGRAMMING) COMPLETE

5.1 4.1 OOP Introduction — Class ani Object

OOP ek programming approach aahe jyat aapan real-world goshtinchaa (jase: Car, Student, Bank Account) code madhe **model** karto.

- **Class** = ek blueprint/design (jase "Car" cha design)
- **Object** = tya blueprint peksha banlelI actual goShTa (jase "tumchi lal Swift car")

```
class Student:
    def __init__(self, name, age):
        self.name = name          # attribute (property)
        self.age = age

    def display(self):            # method (function inside class)
        print(f"Name: {self.name}, Age: {self.age}")

s1 = Student("Rahul", 21)
s2 = Student("Sneha", 20)

s1.display()      # Name: Rahul, Age: 21
s2.display()      # Name: Sneha, Age: 20
```

Important points:

- `__init__` ha ek special method (**constructor**) aahe, jo object banvatAna automatically call hoto.
- `self` mhanje "current object cha reference" — pratyeka method chya pahilyA parameter madhe `self` lihAvecha lagte.
- `s1` ani `s2` he dona vegLae **objects** aahet, prattyekache swtaHche data (`name`, `age`) aahe.

5.2 4.2 Inheritance

Inheritance vaparun ek class (**child class**) dusaRyA class chyA (**parent class**) properties ani methods vaparu shakto — code repeat karnyachI garaj nahi.

```
class Animal:                      # Parent class
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} khAta aahe")

class Dog(Animal):                 # Child class - Animal cha inherit karto
    def bark(self):
        print(f"{self.name} bhaunkat aahe")
```

```
d = Dog("Tommy")
d.eat()       # Tommy khAta aahe   (Parent chi method)
d.bark()      # Tommy bhaunkat aahe (Child chi swtaHchI method)
```

5.2.1 Types of Inheritance

```
# Single Inheritance -> ek parent, ek child (varchya example sarkha)

# Multiple Inheritance -> ek child, dona parents
class Father:
    def skills(self):
        print("Father: Driving, Cooking")

class Mother:
    def talents(self):
        print("Mother: Painting, Singing")

class Child(Father, Mother):
    pass

c = Child()
c.skills()      # Father: Driving, Cooking
c.talents()     # Mother: Painting, Singing

# Multilevel Inheritance -> chain: A -> B -> C
class A:
    def method_a(self):
        print("Class A")

class B(A):
    def method_b(self):
        print("Class B")

class C(B):
    def method_c(self):
        print("Class C")

obj = C()
obj.method_a()   # Class A
obj.method_b()   # Class B
obj.method_c()   # Class C
```

5.2.2 `super()` Function

`super()` vaparun child class madhun parent class chi `__init__` kiMvA method call karta yete.

```python
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)     # Parent cha __init__ call kela
        self.breed = breed

    def display(self):
        print(f"{self.name} is a {self.breed}")

d = Dog("Tommy", "Labrador")
d.display()      # Tommy is a Labrador
```

5.3 4.3 Polymorphism

Polymorphism mhanje "**anek roop**" — same method name vegaLyA classes madhe vegaLyA prakare kaam karto.

```python
class Cat:
    def sound(self):
        print("Meow")

class Cow:
    def sound(self):
        print("Moo")

animals = [Cat(), Cow()]

for animal in animals:
    animal.sound()      # Cat -> Meow, Cow -> Moo (same method call, different output)
```

5.3.1 Method Overriding

Child class, parent class chi method **swtaHchyA pratkare punha define** karto — yAlA "overriding" mhantAta.

```python
class Vehicle:
    def start(self):
        print("Vehicle chalu hote")

class Car(Vehicle):
    def start(self):          # Parent chi method override keli
        print("Car chalu hote key ne")

c = Car()
c.start()      # Car chalu hote key ne
```

5.4 4.4 Encapsulation

Encapsulation mhanje data ani methods la ek class madhe **bandhun (wrap) thevNae**, ani direct access rokNyaasathi private/protected banvaNe. YAmuLe data surakShita rahto.

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # double underscore -> private variable

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance())    # 1500

# print(account.__balance)      # ERROR! Direct access nahi kartA yet (private)
```

Access Modifiers:

Type	Syntax	Access
Public	`self.name`	KuThUnahI access karta yete
Protected	`self._name`	Class ani subclass madhun (convention)
Private	`self.__name`	Fakt tyach class chya aat

5.5 4.5 Abstraction

Abstraction mhanje **complex implementation lpavaun**, phakta garjaeche features user samor dakhavNae. Python madhe abstraction `abc` module vaparun **Abstract Base Class** ne kele jate.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):             # ha method fakt define kela aahe, implementation nahi
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
```

```
    def area(self):              # abstract method la actual implementation dile
        return 3.14 * self.radius * self.radius

class Rectangle(Shape):
    def __init__(self, l, w):
        self.l = l
        self.w = w

    def area(self):
        return self.l * self.w

c = Circle(5)
r = Rectangle(4, 6)
print(c.area())      # 78.5
print(r.area())      # 24

# shape = Shape()    # ERROR! Abstract class cha object banvatA yet nahi
```

Key Point: Abstract class cha directly object banvatA yet nahi — phakta tyachya child classes madhe abstract methods la **actual implementation** dilI jAte.

5.6 4.6 Decorators

Decorator ek function aahe jo dusaRyA function la "**wrap**" karto ani tyacha behavior change/extend karto — original function madhe direct change na karatA.

```
def my_decorator(func):
    def wrapper():
        print("Function suru houn rahilI aahe...")
        func()
        print("Function sampali!")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

Output:

```
Function suru houn rahilI aahe...
Hello!
Function sampali!
```

5.6.1 Practical Example — Timing Decorator

```python
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"Time taken: {end - start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)
    print("Function complete zale")

slow_function()
```

Important: `@decorator_name` syntax vaparun ek function var decorator apply kela jato — he `func = my_decorator(func)` lihNyAchI shortcut aahe.

5.7 4.7 Generators

Generator ek special function aahe jo `return` aivajI **`yield`** vaparto, ani values ek-ek karUna "generate" karto — sagaḷyA values ekach veLai memory madhe store na karatA.

```python
def count_up_to(n):
    count = 1
    while count <= n:
        yield count       # value denato, pan function pause hoto
        count += 1

gen = count_up_to(5)
for num in gen:
    print(num)      # 1 2 3 4 5
```

5.7.1 Generator vs Normal Function — Ka Vaparaycha?

Normal function pura list return karto (sagaḷI memory madhe load hote), pan generator **ek veLai ek value** denato — mhanUna mothya data sathi generator jasta memory-efficient aste.

```python
# Normal way - sagalya squares ekach veLai memory madhe
def get_squares(n):
    result = []
    for i in range(n):
        result.append(i*i)
```

```
    return result

# Generator way - ek-ek value, memory vAchavate
def get_squares_gen(n):
    for i in range(n):
        yield i*i

gen = get_squares_gen(1000000)   # Instant! Kahich memory vaparta nahi ajun
print(next(gen))                 # 0  -> pahili value
print(next(gen))                 # 1  -> pudhchi value
```

5.7.2 Generator Expression

List comprehension sarkhach, pan `()` brackets vaparUna:

```
squares = (x*x for x in range(5))
print(list(squares))    # [0, 1, 4, 9, 16]
```

6 Summary — Pudhe Kay KarAyache?

Abhinandan! Tumhi Python cha saMpUrNa syllabus cover kela — Basics pAsUna OOP concepts parayanta. Ata pudhche steps:

1. **Practice karA** — pratyeka topic cha code svataH type karUna run kara, phakta vachU nakA.
2. **Mini Projects banavA** — jase: Calculator, To-Do List, Number Guessing Game, Student Management System (OOP vaprUna).
3. **Errors ghabraycha nahi** — errors hA learning cha part aahe, Google/ChatGPT/Claude vaparun solve kara.
4. **PudhchA Step** — Python shiklyaanantar Data Structures & Algorithms, Web Development (Django/Flask), kiMvA Data Science/Machine Learning kade jaU shakta.

All the best tumchya Python journey sathi!

7 PART 5: PRACTICE EXERCISES WITH SOLUTIONS

Ha section madhe pratyek topic sathi practice questions ani tyanche solutions dile aahet. Adhi swatah solve karnyacha prayatna kara, mag solution baghA.

7.1 5.1 Variables and Data Types — Practice

Q1. Swap two variables without using a third variable.

```
a = 5
b = 10

a, b = b, a
print(a, b)     # 10 5
```

Q2. Check if a number is int, float, or string using type().

```
value = 45.6
if type(value) == int:
    print("Integer")
elif type(value) == float:
    print("Float")
else:
    print("Other type")
```

Q3. Write a program to take name, age, and city from user and display in one sentence.

```
name = input("Naav: ")
age = input("Vay: ")
city = input("City: ")

print(f"{name} {age} varshacha aahe ani {city} madhe rahto.")
```

7.2 5.2 Operators and Type Casting — Practice

Q1. Find area and perimeter of a rectangle.

```
length = float(input("Length: "))
width = float(input("Width: "))

area = length * width
perimeter = 2 * (length + width)

print("Area:", area)
print("Perimeter:", perimeter)
```

Q2. Check whether a number is even or odd.

```
num = int(input("Number: "))
if num % 2 == 0:
```

```
    print("Even")
else:
    print("Odd")
```

Q3. Convert Celsius to Fahrenheit.

```
celsius = float(input("Celsius: "))
fahrenheit = (celsius * 9/5) + 32
print(f"{celsius}C = {fahrenheit}F")
```

7.3 5.3 Strings — Practice

Q1. Check whether a string is a palindrome (i.e. same forward and backward).

```
s = input("String taka: ")
if s == s[::-1]:
    print("Palindrome aahe")
else:
    print("Palindrome nahi")
```

Q2. Count vowels in a string.

```
s = "Python Programming"
count = 0
for ch in s.lower():
    if ch in "aeiou":
        count += 1
print("Vowels:", count)
```

Q3. Reverse each word in a sentence.

```
sentence = "Python is fun"
words = sentence.split()
reversed_words = [w[::-1] for w in words]
print(" ".join(reversed_words))    # nohtyP si nuf
```

7.4 5.4 Lists, Tuples, Sets, Dictionary — Practice

Q1. Find the largest and smallest number in a list without using max()/min().

```
nums = [23, 45, 12, 67, 34]
largest = nums[0]
smallest = nums[0]

for n in nums:
    if n > largest:
        largest = n
    if n < smallest:
        smallest = n
```

```
print("Largest:", largest)
print("Smallest:", smallest)
```

Q2. Remove duplicates from a list using set.

```
nums = [1, 2, 2, 3, 4, 4, 5]
unique_nums = list(set(nums))
print(unique_nums)     # [1, 2, 3, 4, 5]
```

Q3. Count frequency of each word in a sentence using a dictionary.

```
sentence = "python is easy python is fun"
words = sentence.split()
freq = {}

for word in words:
    if word in freq:
        freq[word] += 1
    else:
        freq[word] = 1

print(freq)   # {'python': 2, 'is': 2, 'easy': 1, 'fun': 1}
```

Q4. Merge two dictionaries.

```
dict1 = {"a": 1, "b": 2}
dict2 = {"c": 3, "d": 4}

merged = {**dict1, **dict2}
print(merged)   # {'a':1,'b':2,'c':3,'d':4}
```

7.5 5.5 Loops and Patterns — Practice

Q1. Print all prime numbers between 1 and 50.

```
for num in range(2, 51):
    is_prime = True
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        print(num, end=" ")
```

Q2. Print a diamond pattern.

```
n = 4
for i in range(1, n+1):
    print(" " * (n-i) + "*" * (2*i-1))
for i in range(n-1, 0, -1):
```

```
    print(" " * (n-i) + "*" * (2*i-1))
```

Q3. Print a multiplication table for a given number.

```
num = 7
for i in range(1, 11):
    print(f"{num} x {i} = {num*i}")
```

7.6 5.6 Functions, Lambda, Recursion — Practice

Q1. Write a function to check if a number is prime.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True

print(is_prime(17))   # True
print(is_prime(15))   # False
```

Q2. Write a lambda function to find the cube of a number.

```
cube = lambda x: x ** 3
print(cube(4))    # 64
```

Q3. Write a recursive function to find sum of digits of a number.

```
def sum_digits(n):
    if n == 0:
        return 0
    return n % 10 + sum_digits(n // 10)

print(sum_digits(1234))   # 10
```

7.7 5.7 File and Exception Handling — Practice

Q1. Count the number of lines and words in a text file.

```
with open("data.txt", "r") as file:
    lines = file.readlines()

line_count = len(lines)
word_count = sum(len(line.split()) for line in lines)

print("Lines:", line_count)
print("Words:", word_count)
```

Q2. Write a program that handles division and asks user to retry on invalid input.

```
while True:
    try:
        a = int(input("First number: "))
        b = int(input("Second number: "))
        print("Result:", a / b)
        break
    except ZeroDivisionError:
        print("Zero ne divide karta yet nahi, punha try kara")
    except ValueError:
        print("Valid number taka!")
```

7.8 5.8 OOP — Practice

Q1. Create a class `Rectangle` with methods to calculate area and perimeter.

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

r = Rectangle(5, 3)
print("Area:", r.area())            # 15
print("Perimeter:", r.perimeter())  # 16
```

Q2. Create a base class `Employee` and a child class `Manager` that adds a bonus.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display(self):
        print(f"{self.name}: Salary = {self.salary}")

class Manager(Employee):
    def __init__(self, name, salary, bonus):
        super().__init__(name, salary)
        self.bonus = bonus

    def display(self):
```

```
        total = self.salary + self.bonus
        print(f"{self.name}: Total Salary = {total}")

m = Manager("Rahul", 50000, 10000)
m.display()    # Rahul: Total Salary = 60000
```

8 PART 6: MINI PROJECTS (FULL CODE)

Ha section madhe chotya real-life projects che complete code dile aahet. He projects vaparun tumhi ek sathi multiple concepts (variables, loops, functions, OOP) practice karu shakta.

8.1 6.1 Project 1 — Simple Calculator

```
def calculator():
    print("=== Simple Calculator ===")
    num1 = float(input("First number: "))
    op = input("Operator (+, -, *, /): ")
    num2 = float(input("Second number: "))

    if op == "+":
        result = num1 + num2
    elif op == "-":
        result = num1 - num2
    elif op == "*":
        result = num1 * num2
    elif op == "/":
        if num2 == 0:
            print("Error: Zero ne divide karta yet nahi")
            return
        result = num1 / num2
    else:
        print("Invalid operator!")
        return

    print(f"Result: {result}")

calculator()
```

8.2 6.2 Project 2 — To-Do List (using Lists)

```
def todo_list():
    tasks = []

    while True:
        print("\n1. Add Task  2. View Tasks  3. Remove Task  4. Exit")
        choice = input("Choose option: ")

        if choice == "1":
            task = input("Enter task: ")
            tasks.append(task)
            print("Task added!")
```

```
        elif choice == "2":
            if not tasks:
                print("No tasks yet!")
            else:
                for i, t in enumerate(tasks, start=1):
                    print(f"{i}. {t}")

        elif choice == "3":
            index = int(input("Task number to remove: ")) - 1
            if 0 <= index < len(tasks):
                removed = tasks.pop(index)
                print(f"Removed: {removed}")
            else:
                print("Invalid task number!")

        elif choice == "4":
            print("Goodbye!")
            break

        else:
            print("Invalid choice!")

todo_list()
```

8.3 6.3 Project 3 — Number Guessing Game

```
import random

def guessing_game():
    number = random.randint(1, 100)
    attempts = 0

    print("Guess the number between 1 and 100!")

    while True:
        guess = int(input("Your guess: "))
        attempts += 1

        if guess < number:
            print("Too low! Try again.")
        elif guess > number:
            print("Too high! Try again.")
        else:
            print(f"Correct! You guessed it in {attempts} attempts.")
            break
```

```
guessing_game()
```

8.4 6.4 Project 4 — Student Management System (Full OOP)

```
class Student:
    def __init__(self, roll_no, name, marks):
        self.roll_no = roll_no
        self.name = name
        self.marks = marks

    def get_grade(self):
        if self.marks >= 90:
            return "A"
        elif self.marks >= 75:
            return "B"
        elif self.marks >= 50:
            return "C"
        else:
            return "Fail"

    def display(self):
        print(f"Roll No: {self.roll_no} | Name: {self.name} | "
              f"Marks: {self.marks} | Grade: {self.get_grade()}")


class StudentManager:
    def __init__(self):
        self.students = []

    def add_student(self, roll_no, name, marks):
        student = Student(roll_no, name, marks)
        self.students.append(student)
        print("Student added successfully!")

    def display_all(self):
        if not self.students:
            print("No students yet!")
        for student in self.students:
            student.display()

    def find_topper(self):
        if not self.students:
            print("No students yet!")
            return
        topper = max(self.students, key=lambda s: s.marks)
        print("Topper is:")
```

```
        topper.display()


# Using the system
manager = StudentManager()
manager.add_student(1, "Rahul", 85)
manager.add_student(2, "Sneha", 92)
manager.add_student(3, "Amit", 67)

print("\n--- All Students ---")
manager.display_all()

print("\n--- Topper ---")
manager.find_topper()
```

Ha project madhe kay shikto:

- **Classes ani Objects** — `Student` class chi individual objects.
- **Encapsulation** — sagaḷa student data ek class madhe organize.
- **Composition** — `StudentManager` class madhe `Student` objects chi list.
- **Lambda + key function** — `max()` sobat topper shodhne.

9 PART 7: COMMON INTERVIEW QUESTIONS (Q&A)

9.1 7.1 Basics

Q: List and Tuple madhe kay farak aahe? A: List mutable (changeable) aahe, Tuple immutable (unchangeable) aahe. List `[]` madhe, Tuple `()` madhe lihla jato. Tuple List peksha fast aste.

Q: == ani `is` operator madhe farak kay? A: == values compare karto (value equal aahe ka), `is` object identity compare karto (donhi ekach memory location var aahet ka).

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)   # True  -> values same aahet
print(a is b)   # False -> vegaLe objects aahet
```

Q: Mutable ani Immutable data types konte aahet? A: Mutable: List, Dictionary, Set. Immutable: Int, Float, String, Tuple, Boolean.

Q: Python madhe `*args` ani `**kwargs` cha upyog kay? A: `*args` function la multiple positional arguments (tuple swaroopat) accept karnyasathi vaparle jate, `**kwargs` multiple keyword arguments (dictionary swaroopat) accept karnyasathi vaparle jate.

9.2 7.2 Data Structures

Q: List Comprehension kashamule fast aste? A: List comprehension internally optimized C-level loop vaparte, tyAmuLe normal `for` loop peksha faster aste ani code pan chota hoto.

Q: Set madhe duplicate values ka nastAta? A: Set internally hash table vaparto, jya madhe pratyek value chi unique hash key aste — duplicate value chi same hash key asalyAmuLe tI automatically ignore hote.

Q: Shallow Copy ani Deep Copy madhe farak kay? A: Shallow copy (`copy.copy()`) fakt top-level object copy karto, andarche nested objects same reference share kartat. Deep copy (`copy.deepcopy()`) sagaḷe nested objects paNa pura wegLe copy karto.

9.3 7.3 Functions and OOP

Q: Function ani Method madhe farak kay? A: Function ek independent block aahe (`def func():`), Method ek function aahe jo class chya aat define kela jato ani object var call kela jato (`obj.method()`).

Q: `__init__` method kashsAThI vaparlA jato? A: `__init__` ek constructor method aahe jo object banvatAna automatically call hoto, ani object chi initial properties (attributes) set karnyasathi vaparlA jato.

Q: Encapsulation ani Abstraction madhe farak kay? A: Encapsulation data ani methods la ek unit madhe wrap karto ani access control karto (private/public). Abstraction implementation details lpavaun fakt necessary features dakhavto. Encapsulation “kase” lapvAyache yAvaar focus karto, Abstraction “kay” dakhvAyache yAvaar.

Q: Class Method, Static Method ani Instance Method madhe farak kay?

```
class Demo:
    def instance_method(self):
        print("Instance method - self vaparto, object chya data la access karto")

    @classmethod
    def class_method(cls):
        print("Class method - cls vaparto, class level data la access karto")

    @staticmethod
    def static_method():
        print("Static method - na self na cls, independent function sarkha")
```

Q: Multiple Inheritance madhe MRO (Method Resolution Order) kay aahe? A: MRO ha order aahe jyAmadhye Python multiple parent classes madhun method search karto (left to right). `ClassName.__mro__` vaparun check kartA yeto.

9.4 7.4 Advanced Concepts

Q: Generator ani Normal Function madhe farak kay? A: Normal function `return` vaparto ani ekach veLai pura result denato. Generator `yield` vaparto ani ek-ek value denato, tyAmuLe memory efficient aste, especially motha data sathi.

Q: Decorator kashamule useful astat? A: Decorators vaparun aapan ek function cha behavior modify/extend karu shakto, original function chya code madhe direct change na karta. Logging, timing, authentication check sarkhya kamansathi khup upyogi.

Q: Python madhe Exception Handling cha `finally` block kadhi vaparaycha? A: `finally` block nehamI execute hoto — exception ala tarihi ani naslA tarihi. File close karne, database connection band karne sarkhya “cleanup” kamansathi vaparlA jato.

10 PART 8: QUICK REFERENCE CHEAT SHEET

10.1 8.1 Common Built-in Functions

Function	Kaam
`print()`	Screen var output dakhawto
`input()`	User kaDUna input ghento
`len()`	Length/size return karto
`type()`	Data type sangto
`int()`, `float()`, `str()`	Type casting
`range()`	Number sequence banvato
`sum()`	Sagalya numbers cha sum
`max()`, `min()`	Maximum/Minimum value
`sorted()`	Sorted list return karto
`len()`	Items/characters count
`enumerate()`	Index sobat item denato
`zip()`	Multiple lists ek sobat combine karto
`map()`	Function pratyek item var apply karto
`filter()`	Condition nusar items filter karto

10.2 8.2 String Methods Quick List

Method	Kaam
`.upper()` / `.lower()`	Capital/small letters
`.strip()`	Extra spaces kadhto
`.split()`	String la list madhe todto
`.join()`	List la string madhe jodto
`.replace()`	Text replace karto
`.find()`	Position shodhto
`.count()`	KitI veLaa aala te sangto
`.startswith()` / `.endswith()`	Suru/shevat check karto

10.3 8.3 List Methods Quick List

Method	Kaam
`.append()`	Shevati item add
`.insert()`	Specific position var add
`.remove()`	Value kadhto
`.pop()`	Index cha item kadhto
`.sort()`	Sort karto
`.reverse()`	Ulta karto
`.index()`	Position sangto
`.count()`	KitI veLaa aala

10.4 8.4 OOP Quick Reference

Concept	Keyword/Syntax
Class banvaNae	`class Name:`
Constructor	`def __init__(self):`
Inheritance	`class Child(Parent):`
Parent method call	`super().method()`
Private variable	`self.__variable`
Abstract class	`from abc import ABC, abstractmethod`
Decorator	`@decorator_name`
Generator	`yield` keyword

10.5 8.5 Exception Handling Quick Syntax

```
try:
    # risky code
    pass
except SomeError:
    # error handle karne
    pass
else:
    # error nasel tar
    pass
finally:
    # nehmi run hoto
    pass
```

11 Final Note

Ha saMpUrNa Python guide 8 parts madhe divide kela aahe — Basics, Data Structures, Functions/Files/Modules, OOP, Practice Exercises, Mini Projects, Interview Q&A, ani Quick Reference. Ha document bajUlA thevaa ani revision sathi vapara. Practice hAcha sarvAta mahatvacha part aahe — jitka jasta code lihAla, titake jasta confident vhal.

Happy Coding!

12 PART 9: BONUS TOPICS (EXTRA DEPTH)

12.1 9.1 Global vs Local Variables

Local variable ek function chya aat define hoto ani fakt tya function madhech vapartA yeto. **Global variable** function chya bahar define hoto ani sagalyA program madhe access kartA yeto.

```
x = 100    # global variable

def show():
    y = 50   # local variable
    print("Inside function:", x, y)

show()
print("Outside function:", x)
# print(y)   # ERROR! y fakt function chya aat vaparta yeto
```

12.1.1 `global` Keyword

Function chya aat global variable la modify karnyasathi `global` keyword vaparava lagto.

```
count = 0

def increment():
    global count
    count += 1

increment()
increment()
print(count)    # 2
```

12.2 9.2 Comments in Python

Comments code explain karnyasathi vaparle jAtAta, ani Python execute kartAna te ignore karto.

```
# Ha ek single line comment aahe
print("Hello")   # ha inline comment

"""
Ha ek multi-line comment/docstring aahe.
Motha explanation lihNyaasathi vaparlA jato.
"""
```

12.3 9.3 The zip() and enumerate() Functions

```
names = ["Rahul", "Sneha", "Amit"]
marks = [85, 92, 78]

# zip() -> multiple lists ek sobat combine karto
for name, mark in zip(names, marks):
    print(f"{name}: {mark}")

# enumerate() -> index sobat item denato
for index, name in enumerate(names):
    print(index, name)
```

12.4 9.4 The Walrus Operator :=

Python 3.8 madhe navIn feature aali — walrus operator vaparun assignment ani condition check ekach line madhe kartA yeto.

```
# Normal way
n = len([1, 2, 3, 4])
if n > 3:
    print(f"List motha aahe, length = {n}")

# Walrus operator way
if (n := len([1, 2, 3, 4])) > 3:
    print(f"List motha aahe, length = {n}")
```

12.5 9.5 Ternary (Conditional) Expression

Ek line madhe if-else lihNyAchI shortcut method.

```
age = 20

# Normal way
if age >= 18:
    status = "Adult"
else:
    status = "Minor"

# Ternary way (ek line madhe)
status = "Adult" if age >= 18 else "Minor"
print(status)
```

12.6 9.6 Common Errors and How to Debug Them

Error	Karan	Solution
`SyntaxError`	Code madhe typing mistake (`:` visarne, spelling)	Line carefully vacha, colon/brackets check kara
`IndentationError`	Spaces/Tabs sahi nahita	4 spaces consistently vapara
`NameError`	Variable define na karatA vaparlA	Variable declare kelay ka check kara
`TypeError`	Chukicha data type operation	`type()` vaparun data type check kara
`IndexError`	List/String cha index range chya bāher	Length check kara (`len()`)
`KeyError`	Dictionary madhe naslelI key vaparli	`.get()` method vapara, direct key nako
`ModuleNotFoundError`	Module install nahi kela	`pip install module_name` karA

Debugging Tips:

1. `print()` statements vaparun variable values check kara — kuthe chuk hote te samjAyalA madat hote.
2. Error message pura vacha — Python nehmi exact line number ani error type sangto.
3. Code la chotya chotya parts madhe todUna test kara.
4. VS Code cha debugger (breakpoints) vapara complex problems sathi.

12.7 9.7 String Formatting Deep Dive

```python
name = "Rahul"
score = 89.5678

# f-string with formatting
print(f"{name} scored {score:.2f} marks")     # 2 decimal places -> 89.57
print(f"{score:>10.2f}")                       # right align, width 10
print(f"{name:^20}")                           # center align, width 20

num = 5
print(f"{num:03d}")     # 005 -> leading zeros

# Percentage formatting
percent = 0.856
print(f"{percent:.1%}")   # 85.6%
```

12.8 9.8 Additional Practice Problems (Mixed)

Q1. Write a program to check if a year is a leap year.

```python
year = int(input("Year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print("Leap year")
else:
    print("Not a leap year")
```

Q2. Find the second largest number in a list.

```python
nums = [45, 12, 89, 34, 89, 23]
unique_sorted = sorted(set(nums), reverse=True)
print("Second largest:", unique_sorted[1])
```

Q3. Check if two strings are anagrams of each other.

```python
def is_anagram(s1, s2):
    return sorted(s1.replace(" ", "").lower()) == sorted(s2.replace(" ", "").lower())

print(is_anagram("listen", "silent"))   # True
print(is_anagram("hello", "world"))     # False
```

Q4. Create a class `BankAccount` with deposit, withdraw, and balance check, handling insufficient funds using exceptions.

```python
class InsufficientFundsError(Exception):
    pass

class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError("Balance kami aahe!")
        self.balance -= amount
        print(f"Withdrew {amount}. New balance: {self.balance}")

account = BankAccount("Rahul", 1000)
account.deposit(500)

try:
    account.withdraw(3000)
except InsufficientFundsError as e:
    print("Error:", e)
```

Q5. Write a generator function that yields Fibonacci numbers up to n terms.

```python
def fibonacci_gen(n):
    a, b = 0, 1
    count = 0
    while count < n:
        yield a
        a, b = b, a + b
        count += 1

for num in fibonacci_gen(10):
    print(num, end=" ")   # 0 1 1 2 3 5 8 13 21 34
```

Q6. Use a decorator to restrict a function to run only if the user is "logged in".

```python
def login_required(func):
    def wrapper(is_logged_in, *args, **kwargs):
        if not is_logged_in:
            print("Access denied! Please login first.")
            return
        return func(*args, **kwargs)
    return wrapper

@login_required
def view_profile():
    print("Welcome to your profile page!")

view_profile(False)   # Access denied!
view_profile(True)    # Welcome to your profile page!
```

13 PART 10: MORE PRACTICE PROBLEMS (ADVANCED SET)

Ha section madhe thoDe jasta challenging problems dile aahet, je multiple concepts ekatra vapartAta.

Q1. Write a program to find all pairs of numbers in a list that sum up to a target value.

```
def find_pairs(nums, target):
    seen = set()
    pairs = []
    for num in nums:
        complement = target - num
        if complement in seen:
            pairs.append((complement, num))
        seen.add(num)
    return pairs

nums = [2, 4, 3, 5, 7, 8, 1]
print(find_pairs(nums, 9))    # [(2,7), (4,5), (8,1)]
```

Q2. Write a program to flatten a nested list.

```
def flatten(nested_list):
    result = []
    for item in nested_list:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

data = [1, [2, 3], [4, [5, 6]], 7]
print(flatten(data))    # [1, 2, 3, 4, 5, 6, 7]
```

Q3. Write a program to check if brackets in an expression are balanced.

```
def is_balanced(expr):
    stack = []
    pairs = {")": "(", "}": "{", "]": "["}

    for ch in expr:
        if ch in "({[":
            stack.append(ch)
        elif ch in ")}]":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return len(stack) == 0
```

```
print(is_balanced("(a+b)*(c-d)"))   # True
print(is_balanced("(a+b*(c-d)"))    # False
```

Q4. Implement a simple Stack using a class.

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if self.is_empty():
            return "Stack is empty!"
        return self.items.pop()

    def peek(self):
        if self.is_empty():
            return "Stack is empty!"
        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0

s = Stack()
s.push(10)
s.push(20)
s.push(30)
print(s.pop())    # 30
print(s.peek())   # 20
```

Q5. Implement a simple Queue using a class.

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if self.is_empty():
            return "Queue is empty!"
        return self.items.pop(0)

    def is_empty(self):
        return len(self.items) == 0
```

```
q = Queue()
q.enqueue(1)
q.enqueue(2)
q.enqueue(3)
print(q.dequeue())    # 1
print(q.dequeue())    # 2
```

Q6. Write a program to find the frequency of each character in a string and display the most common one.

```
from collections import Counter

s = "programming"
freq = Counter(s)
print(freq)                      # Counter({'r':2,'g':2,'m':2, ...})
print(freq.most_common(1))       # [('r', 2)]  or similar
```

Q7. Sort a list of dictionaries by a specific key.

```
students = [
    {"name": "Rahul", "marks": 85},
    {"name": "Sneha", "marks": 92},
    {"name": "Amit", "marks": 78}
]

sorted_students = sorted(students, key=lambda x: x["marks"], reverse=True)
for s in sorted_students:
    print(s["name"], s["marks"])
```

Q8. Write a program to implement binary search.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

nums = [2, 5, 8, 12, 16, 23, 38, 45]
print(binary_search(nums, 23))   # 5 (index)
print(binary_search(nums, 100))  # -1 (not found)
```

Q9. Write a program that implements bubble sort.

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

nums = [64, 34, 25, 12, 22, 11, 90]
print(bubble_sort(nums))   # [11, 12, 22, 25, 34, 64, 90]
```

Q10. Write a class-based implementation of a simple Linked List.

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

ll = LinkedList()
ll.append(10)
ll.append(20)
ll.append(30)
ll.display()    # 10 -> 20 -> 30 -> None
```

14 PART 11: MORE MINI PROJECTS

14.1 11.1 Project 5 — Contact Book (Dictionary + File Storage)

```
import json
import os

FILENAME = "contacts.json"

def load_contacts():
    if os.path.exists(FILENAME):
        with open(FILENAME, "r") as f:
            return json.load(f)
    return {}

def save_contacts(contacts):
    with open(FILENAME, "w") as f:
        json.dump(contacts, f, indent=4)

def contact_book():
    contacts = load_contacts()

    while True:
        print("\n1. Add Contact  2. View Contacts  3. Search  4. Delete  5. Exit")
        choice = input("Choose: ")

        if choice == "1":
            name = input("Name: ")
            phone = input("Phone: ")
            contacts[name] = phone
            save_contacts(contacts)
            print("Contact saved!")

        elif choice == "2":
            for name, phone in contacts.items():
                print(f"{name}: {phone}")

        elif choice == "3":
            name = input("Search name: ")
            print(contacts.get(name, "Not found!"))

        elif choice == "4":
            name = input("Name to delete: ")
            if name in contacts:
                del contacts[name]
                save_contacts(contacts)
                print("Deleted!")
```

```
            else:
                print("Not found!")

        elif choice == "5":
            break

contact_book()
```

14.2 11.2 Project 6 — Quiz Game (OOP + Score Tracking)

```
class Quiz:
    def __init__(self):
        self.questions = [
            {"q": "Capital of India?", "options": ["Mumbai", "Delhi", "Pune"], "answer"
            {"q": "2 + 2 = ?", "options": ["3", "4", "5"], "answer": "4"},
            {"q": "Python is a ___ language?", "options": ["Compiled", "Interpreted", "
        ]
        self.score = 0

    def start(self):
        for item in self.questions:
            print(f"\n{item['q']}")
            for i, opt in enumerate(item["options"], 1):
                print(f"{i}. {opt}")

            answer = input("Your answer: ")
            if answer.strip().lower() == item["answer"].lower():
                print("Correct!")
                self.score += 1
            else:
                print(f"Wrong! Correct answer: {item['answer']}")

        print(f"\nFinal Score: {self.score}/{len(self.questions)}")

quiz = Quiz()
quiz.start()
```

14.3 11.3 Project 7 — Library Management System (Full OOP with Inheritance)

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.is_issued = False
```

```
    def display(self):
        status = "Issued" if self.is_issued else "Available"
        print(f"{self.title} by {self.author} | ISBN: {self.isbn} | Status: {status}")


class Library:
    def __init__(self):
        self.books = []

    def add_book(self, title, author, isbn):
        book = Book(title, author, isbn)
        self.books.append(book)
        print(f"'{title}' added to library.")

    def issue_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                if book.is_issued:
                    print("Book already issued!")
                else:
                    book.is_issued = True
                    print(f"'{book.title}' issued successfully.")
                return
        print("Book not found!")

    def return_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                book.is_issued = False
                print(f"'{book.title}' returned successfully.")
                return
        print("Book not found!")

    def show_all_books(self):
        for book in self.books:
            book.display()


library = Library()
library.add_book("Python Basics", "Guido", "101")
library.add_book("Data Science 101", "Andrew Ng", "102")

library.show_all_books()
library.issue_book("101")
library.show_all_books()
library.return_book("101")
```

14.4 11.4 Project 8 — Inventory Management System

```
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity

    def total_value(self):
        return self.price * self.quantity


class Inventory:
    def __init__(self):
        self.products = {}

    def add_product(self, name, price, quantity):
        if name in self.products:
            self.products[name].quantity += quantity
        else:
            self.products[name] = Product(name, price, quantity)
        print(f"{quantity} units of {name} added.")

    def sell_product(self, name, quantity):
        if name not in self.products:
            print("Product not found!")
            return
        product = self.products[name]
        if product.quantity < quantity:
            print("Not enough stock!")
            return
        product.quantity -= quantity
        print(f"Sold {quantity} units of {name}.")

    def show_inventory(self):
        print("\n--- Inventory ---")
        total_value = 0
        for product in self.products.values():
            value = product.total_value()
            total_value += value
            print(f"{product.name}: Qty={product.quantity}, Price={product.price}, Valu
        print(f"Total Inventory Value: {total_value}")


inv = Inventory()
inv.add_product("Laptop", 50000, 10)
inv.add_product("Mouse", 500, 50)
```

```
inv.sell_product("Laptop", 3)
inv.show_inventory()
```

15 PART 12: USEFUL BUILT-IN MODULES DEEP DIVE

15.1 12.1 `datetime` Module — Dates and Times

```
import datetime

now = datetime.datetime.now()
print(now)                          # current date and time
print(now.year, now.month, now.day) # individual parts

today = datetime.date.today()
print(today)

# Formatting dates
print(now.strftime("%d-%m-%Y"))     # 30-07-2026
print(now.strftime("%H:%M:%S"))     # 14:30:00

# Date difference
d1 = datetime.date(2026, 1, 1)
d2 = datetime.date(2026, 7, 30)
diff = d2 - d1
print(diff.days)                     # number of days between
```

15.2 12.2 `json` Module — Working with JSON Data

```
import json

data = {"name": "Rahul", "age": 21, "skills": ["Python", "SQL"]}

# Convert Python object to JSON string
json_string = json.dumps(data, indent=4)
print(json_string)

# Convert JSON string back to Python object
parsed = json.loads(json_string)
print(parsed["name"])     # Rahul

# Save to file
with open("data.json", "w") as f:
    json.dump(data, f, indent=4)

# Read from file
with open("data.json", "r") as f:
    loaded_data = json.load(f)
    print(loaded_data)
```

15.3 12.3 re Module — Regular Expressions (Basics)

```
import re

text = "My email is rahul123@gmail.com and phone is 9876543210"

# Search for a pattern
email = re.search(r'\S+@\S+', text)
if email:
    print("Email found:", email.group())

# Find all phone numbers (10 digits)
phones = re.findall(r'\d{10}', text)
print("Phones:", phones)

# Replace pattern
masked = re.sub(r'\d{10}', "XXXXXXXXXX", text)
print(masked)

# Validate simple pattern (only letters)
if re.match(r'^[A-Za-z]+$', "Python"):
    print("Valid - only letters")
```

15.4 12.4 os Module — File and Directory Operations

```
import os

print(os.getcwd())              # current working directory

if not os.path.exists("test_folder"):
    os.mkdir("test_folder")     # create a new folder

print(os.listdir("."))          # list files in current directory

print(os.path.exists("data.txt"))   # check if file exists
print(os.path.isfile("data.txt"))   # check if it's a file
```

15.5 12.5 random Module — Random Values

```
import random

print(random.randint(1, 100))          # random int between 1-100
print(random.random())                  # random float between 0-1
print(random.choice(["a", "b", "c"]))   # random item from list

items = [1, 2, 3, 4, 5]
```

```
random.shuffle(items)                    # shuffle list in place
print(items)

print(random.sample(range(1, 50), 6))    # 6 unique random numbers (lottery-style)
```

15.6 12.6 collections Module — Useful Data Structures

```
from collections import Counter, defaultdict, OrderedDict

# Counter - count occurrences
words = ["apple", "banana", "apple", "orange", "banana", "apple"]
count = Counter(words)
print(count)                    # Counter({'apple': 3, 'banana': 2, 'orange': 1})
print(count.most_common(2))     # top 2 most common

# defaultdict - avoid KeyError, gives default value automatically
d = defaultdict(int)
d["a"] += 1
d["a"] += 1
d["b"] += 1
print(d)                        # {'a': 2, 'b': 1}
```

15.7 12.7 itertools Module — Advanced Looping Tools

```
import itertools

# Permutations
for perm in itertools.permutations([1, 2, 3]):
    print(perm)

# Combinations
for combo in itertools.combinations([1, 2, 3, 4], 2):
    print(combo)

# Infinite counter (use with break!)
counter = itertools.count(start=1, step=2)
for i in counter:
    if i > 10:
        break
    print(i, end=" ")     # 1 3 5 7 9
```

16 PART 13: MEGA PROJECT — HOSPITAL MANAGEMENT SYSTEM (Complete OOP)

Ha ek motha project aahe jyAmadhye Inheritance, Encapsulation, Polymorphism, ani Exception Handling — sagaḷe concepts ekatra vaparle aahet.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display_info(self):
        print(f"Name: {self.name}, Age: {self.age}")


class Patient(Person):
    def __init__(self, name, age, disease):
        super().__init__(name, age)
        self.disease = disease
        self.__bill = 0             # private attribute (Encapsulation)

    def add_charge(self, amount):
        self.__bill += amount

    def get_bill(self):
        return self.__bill

    def display_info(self):          # Polymorphism - overriding parent method
        super().display_info()
        print(f"Disease: {self.disease}, Bill: {self.get_bill()}")


class Doctor(Person):
    def __init__(self, name, age, specialization):
        super().__init__(name, age)
        self.specialization = specialization
        self.patients = []

    def add_patient(self, patient):
        self.patients.append(patient)
        print(f"Dr. {self.name} is now treating {patient.name}")

    def display_info(self):           # Polymorphism - different override
        super().display_info()
        print(f"Specialization: {self.specialization}")
        print(f"Number of patients: {len(self.patients)}")
```

```
class AppointmentError(Exception):
    pass


class Hospital:
    def __init__(self, name):
        self.name = name
        self.doctors = []
        self.patients = []

    def add_doctor(self, doctor):
        self.doctors.append(doctor)

    def register_patient(self, patient):
        self.patients.append(patient)

    def book_appointment(self, doctor_name, patient_name):
        doctor = next((d for d in self.doctors if d.name == doctor_name), None)
        patient = next((p for p in self.patients if p.name == patient_name), None)

        if not doctor or not patient:
            raise AppointmentError("Doctor or Patient not found!")

        doctor.add_patient(patient)
        patient.add_charge(500)     # consultation fee

    def show_all(self):
        print(f"\n=== {self.name} Hospital Report ===")
        print("\n-- Doctors --")
        for doc in self.doctors:
            doc.display_info()
            print()

        print("-- Patients --")
        for pat in self.patients:
            pat.display_info()
            print()


# --- Using the system ---
hospital = Hospital("City Care")

dr_mehta = Doctor("Mehta", 45, "Cardiologist")
dr_shah = Doctor("Shah", 38, "Orthopedic")

patient1 = Patient("Rahul", 30, "Chest Pain")
patient2 = Patient("Sneha", 25, "Fracture")
```

```
hospital.add_doctor(dr_mehta)
hospital.add_doctor(dr_shah)
hospital.register_patient(patient1)
hospital.register_patient(patient2)

try:
    hospital.book_appointment("Mehta", "Rahul")
    hospital.book_appointment("Shah", "Sneha")
    hospital.book_appointment("Verma", "Rahul")   # doctor doesn't exist -> error
except AppointmentError as e:
    print("Error:", e)

hospital.show_all()
```

Ha project madhe konte concepts vaparle aahet:

- **Inheritance**: `Patient` ani `Doctor` donhi `Person` cha inherit kartat.
- **Encapsulation**: `Patient` cha `__bill` private attribute aahe, direct access nahi.
- **Polymorphism**: `display_info()` method donhi child classes madhe vegaLyA prakare kaam karto.
- **Custom Exception Handling**: `AppointmentError` vaparun invalid booking handle keli.
- **Composition**: `Hospital` class madhe `Doctor` ani `Patient` objects chi lists.

17 PART 14: QUICK-FIRE QUESTIONS (MCQ & TRUE/FALSE)

Ha section revision sathi khup upyogi aahe — swatah answer dyaycha prayatna kara, mag niche dilele answers check kara.

1. Python madhe variable declare kartAna data type lihAvA lagto. (True/False) Answer: False — Python dynamically typed aahe.

2. List `()` brackets madhe lihlI jate. (True/False) Answer: False — List `[]` madhe, Tuple `()` madhe lihlI jate.

3. `range(5)` kitI numbers denato? a) 4 b) 5 c) 6 Answer: b) 5 (0,1,2,3,4)

4. String immutable aahe. (True/False) Answer: True

5. `for` loop ani `while` loop madhe konta loop condition-based aahe? Answer: `while` loop condition true asel toparyant chalto; `for` loop sequence var iterate karto.

6. `self` keyword cha upyog kay? Answer: `self` current object cha reference denato, class chya methods madhe vaparlA jato.

7. Set madhe duplicate values allowed aahet. (True/False) Answer: False

8. `**kwargs` kashasathi vaparlA jato? Answer: Multiple keyword arguments (dictionary swaroopat) accept karnyasathi.

9. Konta method list la sort karto? a) `.order()` b) `.sort()` c) `.arrange()` Answer: b) `.sort()`

10. Abstract class cha direct object banvatA yeto. (True/False) Answer: False

11. Decorator function kashamule vaparle jAtAta? Answer: Existing function cha behavior modify/extend karnyasathi, code change na karta.

12. Generator `return` vaparto. (True/False) Answer: False — Generator `yield` vaparto.

13. `try-except` block cha `finally` part kadhi run hoto? Answer: Nehmi run hoto — exception ala tarihi ani naslA tarihi.

14. Multiple Inheritance mhanje kay? Answer: Ek child class dona kiMvA jasta parent classes cha inherit karto.

15. `__init__` method la kay mhantAta? Answer: Constructor

16. Dictionary madhe key duplicate asU shakte. (True/False) Answer: False — keys unique astat.

17. `super()` function cha upyoga kay? Answer: Child class chya aatun parent class chi method/constructor call karnyasathi.

18. Lambda function la naav aste. (True/False) Answer: False — Lambda anonymous (nava-naslele) function aste.

19. Exception handling madhe konta block error nsela tarihi run hoto? a) `except` b) `else` c) `finally` Answer: b) `else` (error nsela tarcha run hoto), `finally` mAtra donahi case madhe run hoto.

20. **File read karnyasathi konta mode vaparlA jato?** Answer: `"r"` mode